Prerequisite Practical for Data Structure Labrotary

Title:-

Understanding Basics of Python Programming and Basic Operations on array

Problem statement:-

Understanding Basics of Python Programming and Basic Operations on array i.e. Implement an array with basic operations:

Insert an element

Delete an element

Search for an element

Display the array

Pre-requisite:-

1. Basic knowledge of **Python programming.**

Objective:-

Should be able to develop a Python program of Basics of Python Programming and to implement an array using Python and perform the following basic operations such as Insert an element, Delete an element, Search for an element and Display the element of the array, and

Software and Hardware requirements:-

1. **Operating system:** Linux- Ubuntu 16.04 to 17.10, or Windows 7 to 10,
2. **RAM-** 2GB RAM (4GB preferable)
3. IDE :- Python 3

Outcome:-

students will be able to perform the Basics of Python Programming and create and manipulate arrays efficiently by performing insertion, deletion, searching, and display operations, thereby understanding the fundamental concepts of array data structures.

Theory:-

Data types in Python:-

Answer:- Data types define the type of value a variable can store, such as integers, floating-point numbers, strings, and Boolean values.

basic data types in Python:-

- int – Integer numbers
- float – Decimal numbers
- str – Text/String
- bool – Boolean values (True or False)

variable in Python:-

A variable is a named memory location used to store data.

Variables are created automatically when a value is assigned.

Example:- x = 10 // Integer numbers

name = "John“ // Text/String

Check the data type of a variable:-

- Using the type() function. **Syntax type(variable_name)**
- x = 25
- print(type(x))

String in Python:-

A string is a sequence of characters enclosed in single quotes (' ') or double quotes (" ").

Boolean data type in python**:-**

Boolean stores only two values:

- True
- False

Multiple assignment:-

- Assigning values to multiple variables in one statement.
- a, b, c = 10, 20, 30

Identifiers in Python:-

Names given to variables, functions, classes, or objects.

input function in python:-

- input()
- The input() function always returns the input as a **string (str)**.
- **Example:-**

```
name = input("Enter your name: ")
```

print function in Python**:-**

- print()
- The print() function is used to display output on the screen.
- Example:-

```
print("Hello, World!")
```

Input into an integer:-

- num = int(input("Enter a number: "))

Different ways of string formatting in Python:-

1. Using commas
2. Using + operator
3. Using % formatting
4. Using str.format()
5. Using **f-strings** (recommended)

1. **Using commas :-**

Syntax :- print(value1, value2, value3)

Example:- name = "Alice"

```
age = 20
print("Name:", name, "Age:", age)
```

2. **Using + operator:-**

```
Syntax :- print("Text " + variable)
Example:- name = "Alice"
                print("Hello " + name)
```

3. **Using % formatting:-**

Syntax :- print("format_string" % values)

Common Format Specifiers:-

Specifier	Meaning
%s	String
%d	Integer
%f	Float
%.2f	Float with 2 decimal places

#Example 1: String

```
name = "Alice“
print("Hello %s" % name)
```

#Example 2: Integer

```
marks = 90
print("Marks = %d" % marks)
```

Example 3: Float

```
price = 99.456
print("Price = %.2f" % price)
```

Example 4: Multiple Variables

```
name = "Alice“
age = 20
marks = 89.5
print("Name: %s, Age: %d, Marks: %.1f" % (name, age, marks))
```

4. **Using str.format():-**

Syntax :- print("text {}".format(value))

Example 1:-

```
name = "Alice"
print("Hello {}".format(name))
```

Example 2:-

```
name = "Alice"
age = 20
print("Name: {} Age: {}".format(name, age))
```

Example 3: Named Arguments

```
print("Name: {n}, Age: {a}".format(n="Alice", a=20))
```

5. **sing f-Strings (Recommended)**

An f-string allows variables to be inserted directly into a string.

Syntax :- print(f"text {variable}")

Example 1:-
```
name = "Alice"
print(f'Hello {name}")
```

Example 2:-
```
name = "Alice"
age = 20
print(f"Name: {name}, Age: {age}")
```

Example 3: Expressions
```
a = 10
b = 20
print(f'Sum = {a + b}")
```

Example 4: Decimal Formatting
```
price = 99.456
print(f'Price = {price:.2f}")
```

Control Statements

Control flow is the **heart** of any program.

It is the ability to adjust (or to control) the way that program progresses.

By adjusting the direction of flow programs can become dynamic.

Python supports different control flow statements given below.

1. if statement
2. if-else statement
3.Nested if-else statement
4.If elif else

1.if statement

Syntax:-

if condition:

statement(s)

The if statement to specify a block of python code to be executed if a condition is true

2. if and else

The if..else statement evaluates test expression and will execute body of if only when test condition is True.

If the condition is False, body of else is executed. Indentation is used to separate the blocks

if and else Syntax:-

```
if (condition):
    # Executes this block if
    # condition is true
else:
    # Executes this block if
    # condition is false
```

It executes if block of code if a condition is true and else block if the condition is false

Loops:-

1.The for loop

There is “for in” loop which is similar to for each loop in other languages.

The for loop in Python is used to iterate over a sequence (list, tuple, string) or other iterable objects. Iterating over a sequence is called traversal.

Syntax for Loop **:**

```
for val in sequence:
    Body of for Loop
```

Here, val is the variable that takes the value of the item inside the sequence on each iteration.

Loop continues until we reach the last item in the sequence. The body of for loop is separated from the rest of the code using indentation. It can be used to iterate over iterators and a range

functions:

- Functions in Python programming are self-contained programs that perform some particular tasks.
- Once, a function is created by the programmer for a specific task, this function can be called anytime to perform that task.
- Python gives us many built-in functions like print(), len() etc. but we can also create our own functions. These functions are called user-defined functions.

Function definition:-

- In python, we can use def keyword to define the function.
- The function block is started with the colon (:) and all the same level block statements remain at the same indentation.
- A function can accept any number of parameters that must be the same in the definition and function calling. The syntax to define a function in python is given below.
- def function_name(parameters) :

        statement(s)
        return <expression>

Function calling:-

- In python, a function must be defined before the function calling otherwise the python interpreter gives an error.
- Once the function is defined, we can call it from another function or the python prompt.
- To call the function, use the function name followed by the parentheses.
- Example of calling above function
- Greeting( )
- Example of Defining and Calling function in python:-

```
def Greeting( ):
    print("Hello, Good morning!")
Greeting()
 #Output:- Hello Good morning!
```

Array:-

An **array** is a linear data structure that stores a collection of elements in a sequential order. Each element is accessed using an **index**, where indexing starts from **0**.

In Python, the **list** data type is commonly used to implement arrays because it supports dynamic memory allocation and provides built-in methods for array operations.

Features of an Array:-

1. Stores multiple elements in a single variable.
2. Elements are arranged in contiguous logical order.
3. Supports fast access using indexes.
4. Easy to traverse and manipulate.
5. Suitable for storing homogeneous data (same type of elements).

1. Insertion:-

Insertion means adding a new element to the array at a specified position or at the end of the array.

2. Deletion:-

Deletion removes an existing element from the array.

3. Searching:-

Searching is the process of finding whether a particular element exists in the array.

A **Linear Search** checks each element one by one until the required element is found.

4. Display:-

Display means printing all the elements of the array.

Algorithm:-

1. Perform basic of python programming.
2. Create an array (Python list) with initial elements.
3. Read the element and position for insertion.
4. Insert the element using the insert() method.
5. Read the element to delete.
6. If the element exists, remove it using remove().
7. Read the element to search.
8. Search the element using the in operator and index() method.
9. Display the final array.

Conclusion:-

The experiment demonstrates the implementation of basic of python programming and array operations using Python lists. Operations such as **insertion**, **deletion**, **searching**, and **displaying** were successfully performed. These operations form the foundation for more advanced data structures and algorithms.

Viva Questions:-

1. What is an array?
2. What are the characteristics of an array?
3. What is the index of an array?
4. Why are arrays called linear data structures?
5. What is the difference between an array and a list in Python?
6. What is the time complexity of insertion?
7. What is the time complexity of deletion?
8. What is the time complexity of Linear Search?
9. Which Python data structure is commonly used to implement an array?
10. Give real-life applications of arrays.

Micro Project Based on Prerequisites

- Simple Calculator
- Voting Eligibility Checker
- Currency Converter
- Unit Converter
- Temperature Converter
- Number Guessing Game
- Login Authentication System
- To-Do List Manager
- Fuel Cost Calculator
- Daily Expense Tracker